

BEC305 Python Programming — Module 2 Quiz

Data Structures: Lists, Dictionaries and Strings

Field	Value
Subject code	BEC305
Module	2 — Data Structures: Lists, Dictionaries and Strings
Duration	45 minutes
Maximum marks	50 (Part A 20 + Part B 30)
Attempts allowed	1
Passing marks	20
Textbook	Al Sweigart, Automate the Boring Stuff with Python, 1st ed., 2015

Instructions

- Part A carries 20 one-mark multiple choice questions. Exactly one option is correct.
- Part B carries 13 short questions totalling 30 marks.
- Where a question asks what a snippet prints, give the exact output including brackets and quotes.
- State clearly whether an operation mutates in place or returns a new value.
- No computers or interpreters are permitted during the quiz.

Course outcome and Bloom coverage

Band	Questions	Share	Target
L1–L2 Remember / Understand	7	21%	~30%
L3–L4 Apply / Analyse	24	73%	~45%
L5–L6 Evaluate / Create	2	6%	~25%

Course outcome spread: CO2: 33. Total questions: 33.

Part A — Multiple choice (20 × 1 mark = 20 marks)

A1. For `spam = ['cat', 'bat', 'rat', 'elephant']`, what is `spam[3]`? [CO2 | L2 | 1 mark | easy | Ch.4, pp.80-82]

- (A) 'elephant'
- (B) 'rat'
- (C) `IndexError`
- (D) 'cat'

A2. What is the highest legal index of a list whose `len()` is 4? [CO2 | L2 | 1 mark | easy | Ch.4, pp.80-82]

- (A) 3

- (B) 4
- (C) 5
- (D) It depends on the contents

A3. For `spam = ['cat', 'bat', 'rat', 'elephant']`, what is `spam[1:3]`? [C02 | L3 | 1 mark | medium | Ch.4, pp.82-83]

- (A) `['bat', 'rat']`
- (B) `['bat', 'rat', 'elephant']`
- (C) `['cat', 'bat', 'rat']`
- (D) `'bat'`

A4. Which expression reliably gives the last element of a non-empty list `spam`? [C02 | L2 | 1 mark | easy | Ch.4, pp.82-83]

- (A) `spam[-1]`
- (B) `spam[len(spam)]`
- (C) `spam[0]`
- (D) `spam[len(spam) + 1]`

A5. What is the effect of `del spam[1]` on a four-element list? [C02 | L3 | 1 mark | medium | Ch.4, pp.83-84]

- (A) The element is removed and later elements shift down by one
- (B) The element becomes `None` and the length is unchanged
- (C) The list is emptied
- (D) A new list is returned and `spam` is unchanged

A6. Why is deleting items while looping forward over the same list a bug? [C02 | L4 | 1 mark | hard | Ch.4, pp.83-84]

- (A) Each deletion shifts later elements down, so the loop skips the one that moves into the vacated slot
- (B) Python raises a `RuntimeError` immediately
- (C) Deletion inside a loop is a `SyntaxError`
- (D) The loop restarts from the beginning

A7. What does `spam.sort()` return? [C02 | L2 | 1 mark | medium | Ch.4, pp.89-92]

- (A) `None`
- (B) The sorted list
- (C) A copy of the list
- (D) The number of items sorted

A8. What does `['Alice', 'bob', 'Carol']` become after a plain `sort()`? [C02 | L3 | 1 mark | hard | Ch.4, pp.89-92]

- (A) `['Alice', 'Carol', 'bob']`
- (B) `['Alice', 'bob', 'Carol']`
- (C) `['bob', 'Alice', 'Carol']`
- (D) `['Carol', 'bob', 'Alice']`

A9. Which of these is immutable? [C02 | L1 | 1 mark | easy | Ch.4, pp.93-97]

- (A) tuple
- (B) list
- (C) dictionary

- (D) set

A10. What is the type of (42)? [C02 | L3 | 1 mark | medium | Ch.4, pp.93-97]

- (A) int
- (B) tuple
- (C) list
- (D) SyntaxError

A11. Why can a tuple be a dictionary key when a list cannot? [C02 | L4 | 1 mark | hard | Ch.4, pp.93-97]

- (A) A tuple is immutable and therefore hashable
- (B) A tuple is shorter
- (C) A list is too slow to hash
- (D) Lists cannot be compared for equality

A12. `a = [[1, 2], [3, 4]]` and `b = copy.copy(a)`. After `b[0][0] = 99`, what is `a[0][0]`? [C02 | L4 | 1 mark | hard | Ch.4, pp.97-102]

- (A) 99
- (B) 1
- (C) None
- (D) IndexError

A13. What does `d[key]` do when `key` is not present in dictionary `d`? [C02 | L2 | 1 mark | easy | Ch.5, pp.105-109]

- (A) Raises `KeyError`
- (B) Returns `None`
- (C) Returns an empty string
- (D) Inserts the key with value `None`

A14. What is the difference between `d.get(k, 0)` and `d.setdefault(k, 0)`? [C02 | L3 | 1 mark | medium | Ch.5, pp.109-111]

- (A) `get()` leaves the dictionary unchanged; `setdefault()` inserts the key if it is missing
- (B) They are identical
- (C) `get()` inserts and `setdefault()` does not
- (D) `setdefault()` raises `KeyError` when the key is missing

A15. Why is dictionary lookup preferred over a list scan for large collections? [C02 | L4 | 1 mark | medium | Ch.5, pp.105-109]

- (A) Lookup by key is $O(1)$ on average; scanning a list is $O(n)$
- (B) Dictionaries use less memory
- (C) Lists cannot hold more than 1000 items
- (D) Dictionaries keep the data sorted

A16. `guests = {}`. What happens on `guests['Alice']['apples'] = 5`? [C02 | L4 | 1 mark | hard | Ch.5, pp.113-119]

- (A) `KeyError: 'Alice'`, because the inner dictionary does not exist
- (B) It creates both levels automatically
- (C) It creates a list at the outer level
- (D) `TypeError`

A17. What does `s[0] = 'J'` do when `s` is a string? [C02 | L3 | 1 mark | medium | Ch.6, pp.123-127]

- (A) Raises `TypeError: 'str' object does not support item assignment`
- (B) Changes the first character to 'J'
- (C) Returns a new string
- (D) Raises `IndexError`

A18. What is the value of `"".isdecimal()`? [C02 | L3 | 1 mark | medium | Ch.6, pp.127-131]

- (A) False
- (B) True
- (C) None
- (D) `ValueError`

A19. Which is the correct way to join a list of strings with a comma and space? [C02 | L2 | 1 mark | medium | Ch.6, pp.131-133]

- (A) `','join(items)`
- (B) `items.join(',')`
- (C) `join(items, ',')`
- (D) `items + ', '`

A20. What does `'SpamSpamBaconSpam'.strip('ampS')` return? [C02 | L4 | 1 mark | hard | Ch.6, pp.133-142]

- (A) `'Bacon'`
- (B) `'SpamSpamBaconSpam'`
- (C) `'BaconSpam'`
- (D) `''`

Part B — Descriptive (13 questions, 30 marks)

B1. Explain the difference between indexing a list and slicing it, and why it matters for mutation. [C02 | L4 | 3 marks | Ch.4, pp.80-83]

B2. `a = [[1, 2], [3, 4]]`. Explain what `b = copy.copy(a)` and `c = copy.deepcopy(a)` give you, and demonstrate the difference. [C02 | L4 | 3 marks | Ch.4, pp.97-102]

B3. Explain why the mutating list methods return `None` and what bug that causes. [C02 | L4 | 3 marks | Ch.4, pp.89-92]

B4. Explain when to choose a dictionary over a list, and what you give up by doing so. [C02 | L5 | 3 marks | Ch.5, pp.105-109]

B5. Write a character-frequency counter using `setdefault` and explain why no `if` statement is needed. [C02 | L4 | 2 marks | Ch.5, pp.109-111]

B6. Explain what a tuple is, why it needs a trailing comma when it has one element, and why it can be a dictionary key. [C02 | L3 | 2 marks | Ch.4, pp.93-97]

B7. Explain why deleting from a list while iterating forward over it skips items, and give two correct alternatives. [C02 | L4 | 2 marks | Ch.4, pp.83-87]

B8. Explain the difference between `+=` and `+` when applied to a list, and why it matters. [C02 | L4 | 2 marks | Ch.4, pp.87-89]

B9. Why does a validation loop need a length check as well as an isX test? [CO2 | L3 | 2 marks | Ch.6, pp.127-131]

B10. Explain why join() is called on the separator, and contrast split() with split(' '). [CO2 | L3 | 2 marks | Ch.6, pp.131-133]

B11. Explain why strings are immutable and how an apparent modification is actually performed. [CO2 | L4 | 2 marks | Ch.6, pp.123-127]

B12. Show how to build a nested dictionary safely and explain why.setdefault is needed. [CO2 | L4 | 2 marks | Ch.5, pp.113-119]

B13. Explain what makes a good data model, using a tic-tac-toe board as the example. [CO2 | L5 | 2 marks | Ch.5, pp.111-113]

Answer key — Part A

Q	Answer	CO	Bloom	Reference
A1	A	CO2	L2	Ch.4, pp.80-82
A2	A	CO2	L2	Ch.4, pp.80-82
A3	A	CO2	L3	Ch.4, pp.82-83
A4	A	CO2	L2	Ch.4, pp.82-83
A5	A	CO2	L3	Ch.4, pp.83-84
A6	A	CO2	L4	Ch.4, pp.83-84
A7	A	CO2	L2	Ch.4, pp.89-92
A8	A	CO2	L3	Ch.4, pp.89-92
A9	A	CO2	L1	Ch.4, pp.93-97
A10	A	CO2	L3	Ch.4, pp.93-97
A11	A	CO2	L4	Ch.4, pp.93-97
A12	A	CO2	L4	Ch.4, pp.97-102
A13	A	CO2	L2	Ch.5, pp.105-109
A14	A	CO2	L3	Ch.5, pp.109-111
A15	A	CO2	L4	Ch.5, pp.105-109
A16	A	CO2	L4	Ch.5, pp.113-119
A17	A	CO2	L3	Ch.6, pp.123-127
A18	A	CO2	L3	Ch.6, pp.127-131
A19	A	CO2	L2	Ch.6, pp.131-133
A20	A	CO2	L4	Ch.6, pp.133-142

Part A — why each option is right or wrong

A1. Correct answer: (A) 'elephant'

- (A) Correct - indexes start at 0, so the fourth element sits at index 3.
- (B) Wrong - that is `spam[2]`; counting from 1 is the usual cause of this slip.
- (C) Wrong - index 3 is valid for a four-element list; index 4 would raise.
- (D) Wrong - that is `spam[0]`, the first element.

A2. Correct answer: (A) 3

- (A) Correct - `len` is a count, so the last position is `len - 1`.
- (B) Wrong - using `len` directly as an index raises `IndexError` on every non-empty list.
- (C) Wrong - this is two past the end.
- (D) Wrong - the index range depends only on the length, never on what is stored.

A3. Correct answer: (A) ['bat', 'rat']

- (A) Correct - the slice runs from index 1 up to but not including 3.
- (B) Wrong - this includes index 3, but the upper bound of a slice is excluded.
- (C) Wrong - this starts at 0; the slice starts at the first bound given.
- (D) Wrong - a slice always returns a list, never a bare element.

A4. Correct answer: (A) `spam[-1]`

- (A) Correct - negative indexes count back from the end, with -1 as the last.
- (B) Wrong - this is one past the end and raises `IndexError`.
- (C) Wrong - that is the first element.
- (D) Wrong - this is two past the end.

A5. Correct answer: (A) The element is removed and later elements shift down by one

- (A) Correct - `del` removes in place and the length drops by one.
- (B) Wrong - `del` removes the slot entirely rather than blanking it.
- (C) Wrong - only the named element goes.
- (D) Wrong - `del` is a statement that mutates; it returns nothing.

A6. Correct answer: (A) Each deletion shifts later elements down, so the loop skips the one that moves into the vacated slot

- (A) Correct - the index advances while the contents move the other way, so items are silently missed.
- (B) Wrong - lists permit it without complaint, which is what makes it dangerous.
- (C) Wrong - the syntax is perfectly valid.
- (D) Wrong - the loop continues from where it was, which is precisely the problem.

A7. Correct answer: (A) None

- (A) Correct - it sorts in place and returns `None`, so assigning the result destroys the list.
- (B) Wrong - that is what `sorted(spam)` returns; `sort()` mutates instead.
- (C) Wrong - no copy is made; the original ordering is lost.
- (D) Wrong - no count is returned by any list mutator.

A8. Correct answer: (A) ['Alice', 'Carol', 'bob']

- (A) Correct - sorting is by character code, and every capital precedes every lowercase letter.
- (B) Wrong - that is what `sort(key=str.lower)` gives, not the default.

- (C) Wrong - this is neither ASCII nor case-insensitive order.
- (D) Wrong - that is reverse order, which needs `reverse=True`.

A9. Correct answer: (A) tuple

- (A) Correct - a tuple cannot be changed after creation, which is what makes it hashable.
- (B) Wrong - lists are the archetypal mutable sequence.
- (C) Wrong - dictionaries can gain, lose and change entries.
- (D) Wrong - a set is mutable; `frozenset` is its immutable counterpart.

A10. Correct answer: (A) int

- (A) Correct - the parentheses are only grouping; the comma is what makes a tuple.
- (B) Wrong - a one-element tuple needs a trailing comma, as in `(42,)`.
- (C) Wrong - a list needs square brackets.
- (D) Wrong - the expression parses perfectly well.

A11. Correct answer: (A) A tuple is immutable and therefore hashable

- (A) Correct - a hash must not change over the life of the key, which a mutable value cannot guarantee.
- (B) Wrong - length has nothing to do with it.
- (C) Wrong - speed is not the issue; lists have no hash at all.
- (D) Wrong - lists compare for equality perfectly well.

A12. Correct answer: (A) 99

- (A) Correct - a shallow copy duplicates the outer list only; the inner lists are still shared.
- (B) Wrong - that would require `copy.deepcopy()`, which copies the nested structure too.
- (C) Wrong - the value is an integer throughout.
- (D) Wrong - both indexes are valid.

A13. Correct answer: (A) Raises `KeyError`

- (A) Correct - square-bracket lookup refuses to guess; use `get()` for a default.
- (B) Wrong - `None` is what `get()` returns by default, not square brackets.
- (C) Wrong - no empty value is substituted.
- (D) Wrong - `reading` never inserts; that is `setdefault()`.

A14. Correct answer: (A) `get()` leaves the dictionary unchanged; `setdefault()` inserts the key if it is missing

- (A) Correct - one is for reading with a fallback, the other for initialising.
- (B) Wrong - the difference in whether the dictionary is modified is exactly the point.
- (C) Wrong - this reverses the two behaviours.
- (D) Wrong - avoiding `KeyError` is the reason both methods exist.

A15. Correct answer: (A) Lookup by key is $O(1)$ on average; scanning a list is $O(n)$

- (A) Correct - hashing reaches the value in one step regardless of size.
- (B) Wrong - a dictionary generally uses more memory, not less; speed is the trade.
- (C) Wrong - there is no such limit.
- (D) Wrong - a dictionary expresses no ordering relationship at all.

A16. Correct answer: (A) `KeyError: 'Alice'`, because the inner dictionary does not exist

- (A) Correct - the intermediate level must be created first, typically with `setdefault`.
- (B) Wrong - Python creates no intermediate containers; that is `defaultdict` behaviour.
- (C) Wrong - nothing is created at all.

- (D) Wrong - the failure is a missing key, not a wrong type.

A17. Correct answer: (A) Raises TypeError: 'str' object does not support item assignment

- (A) Correct - strings are immutable, so a change must build a new string.
- (B) Wrong - in-place character assignment is exactly what strings forbid.
- (C) Wrong - the statement raises rather than returning anything.
- (D) Wrong - index 0 is valid for any non-empty string; the problem is assignment.

A18. Correct answer: (A) False

- (A) Correct - every isX method returns False for the empty string, so validate length separately.
- (B) Wrong - an empty string contains no decimal characters, so the test fails.
- (C) Wrong - these methods always return a Boolean.
- (D) Wrong - no exception is raised; the answer is simply False.

A19. Correct answer: (A) ', '.join(items)

- (A) Correct - join is a string method, called on the separator with the list as the argument.
- (B) Wrong - a list has no join method; this raises AttributeError.
- (C) Wrong - there is no such built-in function.
- (D) Wrong - a list and a string cannot be combined with +.

A20. Correct answer: (A) 'Bacon'

- (A) Correct - the argument is a set of characters, and any of them are removed from both ends.
- (B) Wrong - the stripping certainly takes effect here.
- (C) Wrong - stripping works from both ends, not only the front.
- (D) Wrong - B, c, o and n are not in the set, so the middle survives.

Part B — model answers and marking scheme

B1 (3 marks) — CO2, L4, Ch.4, pp.80-83

Question. Explain the difference between indexing a list and slicing it, and why it matters for mutation.

Model answer. An index such as `spam[1]` returns the element itself, so if that element is mutable a change through it is visible to every name referring to the list. A slice such as `spam[1:3]` builds and returns a new list whose upper bound is excluded, so rebinding or replacing that new list leaves the original untouched. The slice is shallow, however: its elements are still references to the same objects, which is why `spam[:]` is not a safe copy for a list of lists.

Point required	Marks
Correct distinction between element and new list	1
Correct statement of the excluded upper bound	1
Correct consequence for mutation, including the shallow-copy caveat	1
Total	3

B2 (3 marks) — CO2, L4, Ch.4, pp.97-102

Question. `a = [[1, 2], [3, 4]]`. Explain what `b = copy.copy(a)` and `c = copy.deepcopy(a)` give you, and demonstrate the difference.

Model answer. `copy.copy(a)` builds a new outer list whose two elements are still references to the same inner lists, so `b` is a shallow copy. `copy.deepcopy(a)` recursively rebuilds the inner lists too, so `c` shares nothing with `a`. Setting `b[0][0] = 99` therefore changes `a[0][0]` to 99 as well, while `c[0][0]` remains 1. The rule is that a slice or `copy.copy` protects the outer level only; nested mutable data needs `deepcopy`, at proportionally higher cost.

Point required	Marks
Correct description of shallow versus deep	1
Correct worked demonstration with the three values	2
Total	3

B3 (3 marks) — CO2, L4, Ch.4, pp.89-92

Question. Explain why the mutating list methods return `None` and what bug that causes.

Model answer. `append()`, `insert()`, `remove()` and `sort()` change the list in place, so there is no new list to hand back and they return `None` by convention, signalling that the work was done to the existing object. The bug this causes is writing `spam = spam.sort()`, which discards the list and binds `spam` to `None`; a subsequent `len(spam)` or index then raises `TypeError`. Either call `spam.sort()` as a statement, or use `spam = sorted(spam)`, which returns a new sorted list and leaves the original alone.

Point required	Marks
Correct explanation of the in-place convention	1
Correct bug identified with both fixes	2
Total	3

B4 (3 marks) — CO2, L5, Ch.5, pp.105-109

Question. Explain when to choose a dictionary over a list, and what you give up by doing so.

Model answer. Choose a dictionary when items are identified by a meaningful label and a list when their position carries meaning, such as a queue order or line numbers. The gain is lookup by key in constant time on average, against a linear scan for a list, which matters enormously as the collection grows. What you give up is any expressed ordering relationship between entries, and the requirement that keys be hashable, so a list cannot serve as a key but a tuple can. A useful test is whether an index in the code needs a comment to explain it; if so, the data wanted a dictionary.

Point required	Marks
Correct criterion of label versus position	1
Correct statement of the complexity gain	1
Correct statement of what is given up	1
Total	3

B5 (2 marks) — CO2, L4, Ch.5, pp.109-111

Question. Write a character-frequency counter using `setdefault` and explain why `no if` statement is needed.

Model answer. Start with `count = {}` and loop over the string: `for ch in text: count[ch] = count.setdefault(ch, 0) + 1`. `setdefault` returns the existing value when the key is present and otherwise inserts the default 0 and returns that, so the first sighting and every repeat are handled by the same line. Without it the code needs an explicit `if ch not in count: count[ch] = 0` before the increment. For 'hello' the result is `{'h':1, 'e':1, 'l':2, 'o':1}`.

Point required	Marks
Correct loop using <code>setdefault</code>	1
Correct explanation of why the <code>if</code> is unnecessary	1
Total	2

B6 (2 marks) — CO2, L3, Ch.4, pp.93-97

Question. Explain what a tuple is, why it needs a trailing comma when it has one element, and why it can be a dictionary key.

Model answer. A tuple is an immutable ordered sequence written with parentheses, suitable for a fixed record such as a coordinate pair. A one-element tuple must be written `(42,)` because the comma, not the parentheses, is what creates a tuple; `(42)` is simply the integer grouped by brackets. Immutability means the value cannot change after creation, so its hash is stable for its lifetime, which is exactly the requirement for a dictionary key. A list has no hash at all and raises `TypeError: unhashable type`.

Point required	Marks
Correct definition and the trailing-comma rule	1
Correct link from immutability to hashability	1
Total	2

B7 (2 marks) — CO2, L4, Ch.4, pp.83-87

Question. Explain why deleting from a list while iterating forward over it skips items, and give two correct alternatives.

Model answer. Deleting at index `i` shifts every later element down by one while the loop index continues to advance, so the element that moves into position `i` is never examined. For a list of four with a deletion at index 1, the original index 2 becomes index 1 and is skipped. Two safe alternatives are iterating over a copy, `for x in spam[:]`, while deleting from `spam`, or building a new list containing only the wanted items, which is usually clearer and is what a list comprehension expresses.

Point required	Marks
Correct explanation of the index-versus-shift mismatch	1
Two valid alternatives given	1
Total	2

B8 (2 marks) — CO2, L4, Ch.4, pp.87-89

Question. Explain the difference between `+=` and `+` when applied to a list, and why it matters.

Model answer. `spam += [3]` extends the existing list in place, so every other name referring to that same list sees the new element. `spam = spam + [3]` evaluates a new list and rebinds only `spam`, leaving any

other name pointing at the original two-element list. The difference is invisible until the list is shared, for example by being passed to a function or assigned to a second name, at which point the in-place version produces an action at a distance that is hard to trace.

Point required	Marks
Correct distinction between in-place extension and rebinding	1
Correct statement of when the difference becomes visible	1
Total	2

B9 (2 marks) — CO2, L3, Ch.6, pp.127-131

Question. Why does a validation loop need a length check as well as an isX test?

Model answer. Every isX method returns False for the empty string, because an empty string contains no characters of the required kind. A program testing only s.isdecimal() therefore cannot distinguish the user typing letters from the user pressing Enter without typing anything, and gives the same misleading message for both. Writing len(s) > 0 and s.isdecimal() separates the two cases, and the order matters because and short-circuits, so the length test runs first.

Point required	Marks
Correct statement that isX returns False for the empty string	1
Correct guard expression with the ordering reason	1
Total	2

B10 (2 marks) — CO2, L3, Ch.6, pp.131-133

Question. Explain why join() is called on the separator, and contrast split() with split(' ').

Model answer. join is a string method, so it is called on the string doing the joining: ', '.join(items). Calling items.join(', ') raises AttributeError because a list has no join method. split() with no argument splits on any run of whitespace and discards empty pieces, so 'a b '.split() gives ['a','b']. split(' ') splits on each single space and keeps the empty pieces between consecutive separators, giving seven elements for the same input, five of them empty.

Point required	Marks
Correct explanation of join being a string method	1
Correct contrast of the two split forms with outputs	1
Total	2

B11 (2 marks) — CO2, L4, Ch.6, pp.123-127

Question. Explain why strings are immutable and how an apparent modification is actually performed.

Model answer. A string cannot be changed after creation, so s[0] = 'J' raises TypeError: 'str' object does not support item assignment. Any apparent modification is performed by building a new string from slices and concatenation and rebinding the name, as in s = 'J' + s[1:]. The consequence is that modifying a string repeatedly inside a loop allocates a new string each pass and costs O(n squared) overall; collecting the parts in a list and calling ''.join(parts) once is the linear-time alternative.

Point required	Marks
Correct statement of immutability with the error	1
Correct rebinding technique and the performance consequence	1
Total	2

B12 (2 marks) — CO2, L4, Ch.5, pp.113-119

Question. Show how to build a nested dictionary safely and explain why.setdefault is needed.

Model answer. With `guests = {}`, the statement `guests['Alice']['apples'] = 5` raises `KeyError: 'Alice'`, because the chained subscript evaluates left to right and the intermediate dictionary does not exist. Calling `guests.setdefault('Alice', {})` creates it if absent and returns it if present, after which `guests['Alice']['apples'] = 5` succeeds. Every intermediate level needs the same treatment; only the innermost assignment can use plain square brackets.

Point required	Marks
Correct identification of the <code>KeyError</code> and its cause	1
Correct <code>setdefault</code> construction for the intermediate level	1
Total	2

B13 (2 marks) — CO2, L5, Ch.5, pp.111-113

Question. Explain what makes a good data model, using a tic-tac-toe board as the example.

Model answer. A good model has the same shape as the thing it describes, so reading the data aloud describes the real object. A board represented as a flat nine-element list forces expressions such as `board[4]`, which needs a comment to explain which square it is and drifts out of step with that comment. Representing it as a dictionary keyed by position name gives `board['mid-M']`, which needs no comment at all. The general test is whether an index requires explanation; if it does, a label was wanted rather than a position.

Point required	Marks
Correct principle of shape matching	1
Correct worked contrast between the two representations	1
Total	2